

PROGETTO PER IL CORSO DI INTELLIGENZA ARTIFICIALE A.A 2023/2024

Giuseppe Magliano 256968 Francesco Dramisino 256718 Alfredo Camposano
256485

Task 1

Il primo punto della relazione, spiega come abbiamo utilizzato il linguaggio PDDL per modellare un problema di pianificazione classico.

Obiettivi del Progetto

Il progetto complessivo prevede:

1. La modellazione di un problema di logistica per fornire quello che viene richiesto dalle diverse workstation.
2. L'implementazione di un algoritmo di ricerca e di un'euristica personalizzati.
3. L'integrazione di un modello temporale in un'architettura di software robotico del mondo reale.

Strumenti Utilizzati

- **PDDL (Planning Domain Definition Language)**: un linguaggio utile per la definizione di domini e problemi di pianificazione, che utilizza predicati e azioni per raggiungere un determinato obiettivo.
- **Libreria PDDL4J**: una libreria che contiene algoritmi di ricerca e varie euristiche, utilizzata insieme a Planutils per eseguire le istanze del dominio e del problema, verificando il funzionamento complessivo.
- **Planutils**: una libreria generale per la configurazione di ambienti basati su Linux, utilizzata per lo sviluppo, l'esecuzione e la valutazione dei pianificatori.
- **Plansys2**: un progetto che fornisce agli sviluppatori di robotica un sistema di pianificazione basato su PDDL, implementato in ROS2, che applica i concetti più recenti della robotica.

Procedura

Abbiamo iniziato dalla modellazione del dominio e del problema, affrontando il punto 1 del progetto, composto da due istanze. Rispetto ai requisiti imposti, abbiamo modellato il seguente dominio del problema:

```
(define (domain dominio)
  (:requirements :typing :existential-preconditions
    :negative-preconditions :disjunctive-preconditions )
  (:types
    location locatable - object
    box agent carrier workstation content - locatable
    quantity - object
  )
  (:predicates
    (box-vuota ?b - box)
    (capacita ?c - carrier ?cn - quantity)
    (pos ?x - locatable ?v - location)
    (nella-scatola ?b - box ?ct - content)
    (servito ?p - workstation ?c - content)
    (sul-carrello ?b - box ?c - carrier)
    (carrello-vuoto ?c - carrier)
    (precedente ?cn1 ?cn2 - quantity)
  )
  (:action muovi
    :parameters (?a - agent ?l1 ?l2 - location ?c - carrier)
    :precondition (and
      (pos ?c ?l1)
      (pos ?a ?l1)
      (not (carrello-vuoto ?c))
    )
    :effect (and
      (not (pos ?c ?l1))
      (not (pos ?a ?l1))
      (pos ?c ?l2)
      (pos ?a ?l2)
    )
  )
  (:action muovi-agente
    :parameters (?a - agent ?l1 ?l2 - location)
    :precondition (and
      (pos ?a ?l1)
    )
    :effect (and
      (not (pos ?a ?l1))
      (pos ?a ?l2)
    )
  )
)
```

```

    )
  )

  (:action prendi
    :parameters (?a - agent ?l - location ?c - carrier ?b - box ?q1
?q2 - quantity)
    :precondition (and
      (pos ?c ?l)
      (pos ?a ?l)
      (pos ?b ?l)
      (capacita ?c ?q2)
      (precedente ?q1 ?q2)
    )
    :effect (and
      (not (pos ?b ?l))
      (capacita ?c ?q1)
      (sul-carrello ?b ?c)
      (not (capacita ?c ?q2))
      (not (carrello-vuoto ?c))
    )
  )

  (:action toglì
    :parameters (?a - agent ?l - location ?c - carrier ?b - box ?q1
?q2 - quantity)
    :precondition (and
      (pos ?c ?l)
      (pos ?a ?l)
      (capacita ?c ?q1)
      (sul-carrello ?b ?c)
      (precedente ?q1 ?q2)
    )
    :effect (and
      (pos ?b ?l)
      (capacita ?c ?q2)
      (not (capacita ?c ?q1))
      (not (sul-carrello ?b ?c))
    )
  )

  (:action riempi
    :parameters (?a - agent ?l - location ?c - content ?b - box)
    :precondition (and
      (pos ?a ?l)
      (pos ?c ?l)
      (box-vuota ?b)
      (or (exists (?car - carrier)

```

```

        (and
        (pos ?car ?l)
        (sul-carrello ?b ?car)
        )
    )
    (pos ?b ?l)
)
)
:effect (and
(not (box-vuota ?b))
(nella-scatola ?b ?c)
)
)
(:action svuota
  :parameters (?a - agent ?l - location ?c - content ?b - box ?p -
workstation)
  :precondition (and
    (pos ?a ?l)
    (pos ?p ?l)
    (not (box-vuota ?b))
    (nella-scatola ?b ?c)
    (or (exists (?car - carrier)
      (and
        (pos ?car ?l)
        (sul-carrello ?b ?car)
      )
    )
    (pos ?b ?l)
  )
  :effect (and
    (box-vuota ?b)
    (servito ?p ?c)
    (not (nella-scatola ?b ?c))
  )
)
)
)

```

Per prima cosa abbiamo definito i tipi degli oggetti interessati nel nostro dominio. Abbiamo due tipi di oggetti principali:

- **Object:**
 - **location:** rappresenta le locazioni in cui si trovano le workstation.
 - **locatable:** sono gli oggetti che possono essere allocati.
 - **quantity:** rappresenta la quantità di spazio disponibile nel carrello.

Abbiamo anche i tipi derivati da **locatable**:

- **box**: rappresenta le scatole che devono essere trasportate.
- **agent**: rappresenta l'agente robotico che trasporta le scatole.
- **carrier**: il carrello utilizzato dall'agente robotico.
- **workstation**: le stazioni di lavoro che devono essere servite.
- **content**: il contenuto generico delle scatole (valvole, bulloni, attrezzi).

Predicati

Successivamente, abbiamo definito i vari predicati necessari:

- **(box-vuota ?b - box)**: indica che la scatola b è vuota.
- **(capacita ?ca - carrier ?q - quantity)**: indica la capacità attuale del carrello.
- **(pos ?lb - locatable ?lc - location)**: indica che un oggetto allocabile lb si trova nella posizione lc.
- **(nella-scatola ?b - box ?ct - content)**: indica che un contenuto ct è all'interno della scatola b.
- **(servito ?ws - workstation ?ct - content)**: indica che una workstation ws è stata servita con un contenuto ct.
- **(sul-carrello ?b - box ?ca - carrier)**: indica che una scatola b è sul carrello ca.
- **(carrello-vuoto ?ca - carrier)**: indica che il carrello ca è vuoto.
- **(precedente ?q1 ?q2 - quantity)**: questo predicato ci permette di definire che q1 è una quantità precedente a q2, utile per incrementare e decrementare lo spazio disponibile nel carrello.

Azioni

Dopo la creazione dei predicati, siamo passati alla definizione delle azioni:

Muovi

- **Descrizione**: Permette di spostare sia l'agente robotico che il carrier da una locazione a un'altra.
- **Parametri**: Un agente, un carrier, e due locazioni (di partenza e di destinazione).
- **Precondizioni**: Il carrier e l'agente devono trovarsi nella stessa locazione iniziale (I1) e il carrier non deve essere vuoto.
- **Effetti**: Il carrier e l'agente non sono più presenti nella locazione iniziale (I1), ma si trovano nella locazione di destinazione (I2).

Muovi-Agente

- **Descrizione**: Permette di spostare solo l'agente senza il carrier.
- **Parametri**: Un agente e due locazioni (di partenza e di destinazione).
- **Precondizioni**: L'agente deve trovarsi nella locazione iniziale (I1).
- **Effetti**: L'agente non è più presente nella locazione iniziale (I1), ma si trova nella locazione di destinazione (I2).

Prendi

- **Descrizione:** Permette di prendere una scatola e caricarla sul carrier.
- **Parametri:** Un agente, un carrier, una locazione, una scatola e due quantità (q_1 e q_2) per gestire la capacità del carrier.
- **Precondizioni:** L'agente, il carrier e la scatola devono trovarsi nella stessa locazione (l). La capacità attuale del carrier deve essere q_2 e deve esistere una relazione di precedenza tra q_1 e q_2 .
- **Effetti:** La scatola non si trova più nella locazione (l), la capacità del carrier viene decrementata a q_1 , la scatola è ora sul carrier, il carrier non è vuoto e la capacità del carrier non è più q_2 .

Riempi

- **Descrizione:** Permette di riempire una scatola con un contenuto.
- **Parametri:** Un agente, una locazione, un contenuto e una scatola.
- **Precondizioni:** L'agente e il contenuto devono trovarsi nella stessa locazione (l). La scatola deve essere vuota e può trovarsi sia sul carrier sia direttamente nella locazione.
- **Effetti:** La scatola non è più vuota e contiene il contenuto specificato.

Svuota

- **Descrizione:** Permette di svuotare una scatola e servire il contenuto a una workstation.
- **Parametri:** Un agente, una locazione, un contenuto, una scatola e una workstation.
- **Precondizioni:** L'agente e la workstation devono trovarsi nella stessa locazione (l). La scatola non deve essere vuota e deve contenere il contenuto specificato.
- **Effetti:** La scatola diventa vuota, il contenuto è stato servito alla workstation, e non si trova più nella scatola.

Togli

- **Descrizione:** Permette di togliere una scatola dal carrier.
- **Parametri:** Un agente, una locazione, un carrier, una scatola e due quantità (q_1 e q_2).
- **Precondizioni:** L'agente e il carrier devono trovarsi nella stessa locazione (l). La capacità attuale del carrier deve essere q_1 , la scatola deve essere sul carrier, e deve esistere una relazione di precedenza tra q_1 e q_2 .
- **Effetti:** La scatola si trova ora nella locazione (l), la capacità del carrier viene incrementata a q_2 , e la scatola non è più sul carrier.

Per quanto riguarda il problema andiamo ad istanziare le variabili che sono richieste dalla traccia, in questo caso nell'istanza 1 abbiamo:

```

(define (problem istanza1)
  (:domain dominio)
  (:objects
    a1 - agent
    c1 - carrier
    b1 b2 b3 - box
    q0 q1 - quantity
    bolt valve tool - content
    ws1 ws2 ws3 ws4 - workstation
    l1 l2 l3 l4 warehouse - location
  )
  (:init
    (pos ws1 l1)
    (pos ws2 l1)
    (pos ws3 l2)
    (pos ws4 l3)
    (box-vuota b1)
    (box-vuota b2)
    (box-vuota b3)
    (capacita c1 q1)
    (precedente q0 q1)
    (pos a1 warehouse)
    (pos c1 warehouse)
    (pos b1 warehouse)
    (pos b2 warehouse)
    (pos b3 warehouse)
    (carrello-vuoto c1)
    (pos bolt warehouse)
    (pos tool warehouse)
    (pos valve warehouse)
  )
  (:goal
    (and
      (servito ws1 tool)
      (servito ws1 bolt)
      (servito ws2 bolt)
      (servito ws3 valve)
      (servito ws4 tool)
      (servito ws4 valve)
    )
  )
)

```

un solo agent

5 luoghi (compreso il magazzino)

3 scatole

un solo carrier

come contenuto abbiamo una quantità generica di bolt, valve e tool

4 workstation e 1 posto disponibili sul carrier

In seguito, definiamo i predicati che dovranno essere veri inizialmente, anche questi dedotti dalla traccia, in particolare sappiamo che agent, carrier, bolt, valve, tool e tutte le scatole si trovano inizialmente al deposito, sappiamo anche che tutte le scatole sono vuote come anche i posti sul carrier, e la capacità del carrier è 1. Inoltre definiamo le posizioni delle workstation.

Infine il goal da soddisfare è quello di consegnare tool, bolt e valve alle diverse workstation in base alla richiesta.

Una volta creati i file di dominio e problema, abbiamo utilizzato Planutils per runnare il tutto e farci generare il piano, tramite i comandi:

- **Planutils install dual-bfws bfws-ffparser** che è un package all'interno di planutils
- **Planutils run dual-bfws- ffparser domain.pddl ist1.pddl**

Come possiamo vedere alla fine è stato trovato il piano e il tempo impiegato è stato 0.54 secondi ed è stato stampato a schermo.

Found plan as follows:

```
00: (prendi a1 warehouse c1 b3 q0 q1) [0]
01: (   riempi a1 warehouse valve b3) [0]
02: (       muovi a1 warehouse l2 c1) [0]
03: (       svuota a1 l2 valve b3 ws3) [0]
04: (       muovi a1 l2 warehouse c1) [0]
05: (   riempi a1 warehouse valve b3) [0]
06: (       muovi a1 warehouse l3 c1) [0]
07: (       svuota a1 l3 valve b3 ws4) [0]
08: (       muovi a1 l3 warehouse c1) [0]
09: ( riempi a1 warehouse tool b3) [0]
10: (       muovi a1 warehouse l3 c1) [0]
11: (       svuota a1 l3 tool b3 ws4) [0]
12: (       muovi a1 l3 warehouse c1) [0]
13: ( riempi a1 warehouse tool b3) [0]
14: (       muovi a1 warehouse l1 c1) [0]
15: (       svuota a1 l1 tool b3 ws1) [0]
16: (       muovi a1 l1 warehouse c1) [0]
17: ( riempi a1 warehouse bolt b3) [0]
18: (       muovi a1 warehouse l1 c1) [0]
```

```
19: (      svuota a1 l1 bolt b3 ws2) [0]
20: (      muovi a1 l1 warehouse c1) [0]
21: ( riempi a1 warehouse bolt b3) [0]
22: (      muovi a1 warehouse l1 c1) [0]
23: (      svuota a1 l1 bolt b3 ws1) [0]
```

Time spent **for** searching: 0,54 seconds

Nodes Explored: 2017

Memory used **for** search: -2,38 MBytes

Utilizzando lo stesso dominio e variando le inizializzazioni del problema, abbiamo implementato anche l'istanza 2:

```
(define (problem istanza2)
  (:domain dominio)
  (:objects
    a1 a2 - agent
    c1 c2 - carrier
    b1 b2 b3 - box
    q0 q1 q2 - quantity
    bolt valve tool - content
    ws1 ws2 ws3 ws4 ws5 ws6 - workstation
    l1 l2 l3 l4 warehouse - location
  )

  (:init
    (pos ws1 l1)
    (pos ws2 l1)
    (pos ws3 l2)
    (pos ws4 l3)
    (pos ws5 l3)
    (pos ws6 l4)
    (box-vuota b1)
    (box-vuota b2)
    (box-vuota b3)
    (capacita c1 q2)
    (capacita c2 q2)
    (precedente q0 q1)
    (precedente q1 q2)
    (pos a1 warehouse)
    (pos a2 warehouse)
    (pos c1 warehouse)
    (pos c2 warehouse)
    (pos b1 warehouse)
    (pos b2 warehouse)
    (pos b3 warehouse)
```

```

        (carrello-vuoto c1)
        (carrello-vuoto c2)
        (pos bolt warehouse)
        (pos tool warehouse)
        (pos valve warehouse)
    )

    (:goal
      (and
        (servito ws1 tool)
        (servito ws1 bolt)
        (servito ws2 bolt)
        (servito ws3 valve)
        (servito ws4 tool)
        (servito ws4 valve)
        (servito ws5 valve)
        (servito ws5 bolt)
        (servito ws5 tool)
        (servito ws6 valve)
        (servito ws6 bolt)
        (servito ws6 tool)
      )
    )
  )
)

```

In questo caso nel problema avremo un obiettivo più complesso perché le istanze sono più che raddoppiate, infatti avremo:

2 robot

2 carrier

5 diverse location tra cui il warehouse

3 scatole

6 workstation da soddisfare

2 posti per ogni carrier

La parte di inizializzazione rimane identica se non per il fatto che devono essere inizializzati i nuovi oggetti aggiunti.

Questa volta, a differenza della precedente, il costo del piano è aumentato, in quanto sono presenti più vincoli da soddisfare ma anche il tempo per generarlo è aumentato, ci sono voluti 90 secondi per generare il seguente piano.

Found plan as follows:

```
00: (prendi a1 warehouse c1 b1 q1 q2) [0]
01: (   riempi a1 warehouse valve b3) [0]
02: (prendi a1 warehouse c1 b3 q0 q1) [0]
03: (   riempi a1 warehouse bolt b1) [0]
04: (       muovi a1 warehouse l2 c1) [0]
05: (       svuota a1 l2 valve b3 ws3) [0]
06: (           muovi a1 l2 l1 c1) [0]
07: (       svuota a1 l1 bolt b1 ws2) [0]
08: (       muovi a1 l1 warehouse c1) [0]
09: (   riempi a1 warehouse tool b1) [0]
10: (   riempi a1 warehouse bolt b3) [0]
11: (       muovi a1 warehouse l1 c1) [0]
12: (       svuota a1 l1 tool b1 ws1) [0]
13: (       svuota a1 l1 bolt b3 ws1) [0]
14: (       muovi a1 l1 warehouse c1) [0]
15: (   riempi a1 warehouse bolt b3) [0]
16: (   riempi a1 warehouse valve b1) [0]
17: (       muovi a1 warehouse l3 c1) [0]
18: (       svuota a1 l3 bolt b3 ws5) [0]
19: (       svuota a1 l3 valve b1 ws4) [0]
20: (       muovi a1 l3 warehouse c1) [0]
21: (   riempi a1 warehouse bolt b1) [0]
22: (       muovi a1 warehouse l4 c1) [0]
23: (       svuota a1 l4 bolt b1 ws6) [0]
24: (       muovi a1 l4 warehouse c1) [0]
25: (   riempi a1 warehouse valve b1) [0]
26: (   riempi a1 warehouse tool b3) [0]
27: (       muovi a1 warehouse l4 c1) [0]
28: (       svuota a1 l4 valve b1 ws6) [0]
29: (       svuota a1 l4 tool b3 ws6) [0]
30: (       muovi a1 l4 warehouse c1) [0]
31: (   riempi a1 warehouse valve b1) [0]
32: (   riempi a1 warehouse tool b3) [0]
33: (       muovi a1 warehouse l3 c1) [0]
34: (       svuota a1 l3 valve b1 ws5) [0]
35: (prendi a2 warehouse c2 b2 q1 q2) [0]
36: (       svuota a1 l3 tool b3 ws4) [0]
37: (   riempi a2 warehouse tool b2) [0]
38: (       muovi a2 warehouse l3 c2) [0]
39: (       svuota a1 l3 tool b2 ws5) [0]
```

Time spent for searching: 89,93 seconds

Nodes Explored: 259352

Memory used for search: 32,06 MBytes

Task 2

Nel secondo punto della traccia viene chiesto di implementare un proprio algoritmo di ricerca e le relative euristiche per risolvere le istanze del problema fornite.

Algoritmi di Ricerca

Per sviluppare un pianificatore capace di trovare soluzioni adeguate per le istanze fornite, è stata utilizzata la libreria PDDL4J. Sono state implementate diverse tecniche di ricerca e euristiche per risolvere problemi di pianificazione.

Abbiamo creato una classe denominata Algoritmi che contiene gli algoritmi di ricerca e le euristiche disponibili.

Algoritmi Utilizzati

A*

Inizialmente, abbiamo sperimentato l'algoritmo A*, noto per la sua capacità di trovare piani ottimali quando usato con un'euristica ammissibile.

Weighted A*

Successivamente, ci siamo concentrati su algoritmi che potessero generare piani sub-ottimali in tempi ragionevoli. Abbiamo adottato l'algoritmo Weighted A*, una variante di A* in cui l'euristica di ciascun nodo viene moltiplicata per un peso. In questo algoritmo, per un nodo n la funzione di costo è:

$$f(n) = \text{peso} \times h(n) + g(n)$$

dove $g(n)$ rappresenta il costo effettivo per raggiungere n , $h(n)$ è la stima del costo per arrivare al goal, e $f(n)$ è la funzione di costo totale.

Il codice dell'algoritmo Weighted A*, implementa questa tecnica di ricerca per la pianificazione. Inizia definendo variabili per gestire il peso w , il timeout, il numero di nodi esplorati, la memoria utilizzata e il tempo di ricerca. Il costruttore della classe carica e istanzia il problema da file, inizializzando anche il tipo di euristica e di ricerca.

Il metodo **search** esegue l'algoritmo utilizzando una coda di priorità per gestire i nodi da esplorare. Ogni nodo viene valutato con la funzione $f(n)$. Durante la ricerca, i nodi vengono estratti dalla coda di priorità in base al valore di $f(n)$. Se un nodo raggiunge il goal, la ricerca si conclude restituendo il percorso trovato. Altrimenti, i successori del nodo vengono generati e inseriti nella coda di priorità. L'algoritmo continua fino a trovare il percorso ottimale o esaurire il tempo disponibile. La funzione **instantiate** crea e inizializza un oggetto **Problem**, mentre **isSupported** verifica la compatibilità del problema con l'algoritmo.

```

public Plan weighted_astar(Problem codedProblem) throws ProblemNotSupportedException {
    validateProblem(codedProblem);
    logSearchStartAStar();

    long startTime = System.currentTimeMillis();
    long initialMemory = getCurrentMemoryUsed();

    StateHeuristic heuristic = createHeuristic(heuristicType, codedProblem);
    State initState = new State(codedProblem.getInitialState());

    Map<State, Node> openSet = new HashMap<>();
    Map<State, Node> closedSet = new HashMap<>();

    PriorityQueue<Node> openQueue = createPriorityQueue(this.weight);
    Node rootNode = new Node(initState, (Node) null, -1, 0.0, (double) heuristic.estimate(initState, codedProblem.getGoal()));

    openQueue.add(rootNode);
    openSet.put(initState, rootNode);

    this.searchingTime = 0L;
    this.memoryUsed = 0L;
    this.exploredNodes = 0;
    Node goal = null;
    long timeout = minutesToMilliseconds(this.timeout);
    double bestCost = Integer.MAX_VALUE;
    long searchingTime;

    for (searchingTime = 0L; !openQueue.isEmpty() && searchingTime < timeout; searchingTime = System.currentTimeMillis() - startTime) {
        Node current = (Node) openQueue.poll();
        openSet.remove(current);
        closedSet.put(current, current);

        if (current.satisfy(codedProblem.getGoal())) {
            goal = current;
            break;
        }
        int index = 0;

        for (Iterator i = codedProblem.getActions().iterator(); i.hasNext(); index++) {
            Action action = (Action) i.next();
            if (action.isApplicable(current)) {
                Node state = applyEffects(current, action);
                double cost = current.getCost() + action.getCost().getValue();
                Node result = (Node) openSet.get(state);
                if (result == null) {
                    result = (Node) closedSet.get(state);
                    if (result != null) {
                        if (cost < result.getCost()) {
                            result.setCost(cost);
                            result.setParent(current);
                            result.setAction(index);
                            result.setDepth(current.getDepth() + 1);
                            openQueue.add(result);
                            openSet.put(result, result);
                            closedSet.remove(result);
                        }
                    } else {
                        state.setCost(cost);
                        state.setParent(current);
                        state.setAction(index);
                        state.setHeuristic(heuristic.estimate(state, codedProblem.getGoal()));
                        state.setDepth(current.getDepth() + 1);
                        openQueue.add(state);
                        openSet.put(state, state);
                    }
                } else if (cost < result.getCost()) {
                    result.setCost(cost);
                    result.setParent(current);
                    result.setAction(index);
                    result.setDepth(current.getDepth() + 1);
                }
            }
        }
    }

    long currentMemory = getCurrentMemoryUsed();
    this.memoryUsed = currentMemory - initialMemory;
    this.searchingTime = searchingTime;
    this.exploredNodes = closedSet.size();

    if (goal == null) { return null; }
    else {
        Node n = goal;
        SequentialPlan plan;
        for (plan = new SequentialPlan(); n.getParent() != null; n = n.getParent()) {
            Action action = (Action) codedProblem.getActions().get(n.getAction());
            plan.add(0, action);
        }
        return plan;
    }
}

```

Enforced Hill Climbing

L'algoritmo Enforced Hill Climbing (EHC) inizia dallo stato iniziale, calcolando il valore dell'euristica per questo stato. Successivamente, cerca il miglior successore dello stato corrente, minimizzando il valore dell'euristica; se trova un successore con un valore migliore, l'algoritmo si sposta in questo nuovo stato e ripete il processo. Se non viene trovato un successore migliore, l'algoritmo avvia una ricerca in ampiezza limitata (BFS) dallo stato corrente, continuando fino a trovare uno stato con un valore dell'euristica migliore. L'algoritmo quindi si sposta nello stato con il miglior valore dell'euristica trovato dalla BFS e riprende il processo di Hill Climbing. L'algoritmo termina quando raggiunge lo stato obiettivo o quando non è possibile migliorare ulteriormente il valore dell'euristica.

Il codice dell'algoritmo Enforced Hill Climbing implementa questa tecnica di ricerca per la pianificazione. Inizia definendo variabili per gestire lo stato iniziale, l'euristica, e i successori degli stati. Il costruttore della classe carica e istanzia il problema da file, inizializzando anche il tipo di euristica da utilizzare.

Il metodo **search** esegue l'algoritmo iniziando dalla valutazione dell'euristica dello stato iniziale. L'algoritmo cerca iterativamente il miglior successore minimizzando il valore dell'euristica. Se un successore con un valore migliore viene trovato, l'algoritmo si sposta in questo nuovo stato e ripete il processo. Se non ci sono successori con un valore migliore, l'algoritmo avvia una ricerca BFS limitata dallo stato corrente per trovare uno stato con un'euristica migliore. Questo processo continua finché non viene trovato lo stato obiettivo o non si può più migliorare il valore dell'euristica.

La funzione **instantiate** crea e inizializza un oggetto **Problem**, mentre **isSupported** verifica la compatibilità del problema con l'algoritmo.

```

public Plan enforced_hillclimbing(Problem codedProblem) throws ProblemNotSupportedException {
    validateProblem(codedProblem);

    long startTime = System.currentTimeMillis();
    long initialMemory = getCurrentMemoryUsed();

    StateHeuristic heuristic = createHeuristic(heuristicType, codedProblem);
    LinkedList<Node> list = new LinkedList<>();
    State initState = new State(codedProblem.getInitialState());
    Node rootNode = new Node(initState, (Node) null, 0, 0.0, (double) heuristic.estimate(initState,
        codedProblem.getGoal()));

    list.add(rootNode);
    double bestHeuristic = rootNode.getHeuristic();

    this.searchingTime = 0L;
    this.memoryUsed = 0L;
    this.exploredNodes = 0;
    Node goal = null;
    long timeout = minutesToMilliseconds(this.timeout);
    boolean deadEndFree = true;

    long searchingTime;
    long end;
    for (searchingTime = 0L; !list.isEmpty() && goal == null && deadEndFree
        && searchingTime < timeout; searchingTime = end - startTime) {

        Node current = (Node) list.pop();
        this.exploredNodes++;

        LinkedList<Node> successors = this.getSuccessors(current, codedProblem, heuristic);

        Node successor;
        for (deadEndFree = !successors.isEmpty(); !successors.isEmpty() && goal == null; list
            .addLast(successor)) {
            successor = (Node) successors.pop();

            double heuristicSuccessor = successor.getHeuristic();
            if (heuristicSuccessor == 0.0) {
                goal = successor;
            }

            if (heuristicSuccessor < bestHeuristic) {
                successors.clear();
                list.clear();
                bestHeuristic = heuristicSuccessor;
            }
        }
        end = System.currentTimeMillis();
    }

    long currentMemory = getCurrentMemoryUsed();
    this.memoryUsed = currentMemory - initialMemory;
    this.searchingTime = searchingTime;
    if (goal == null) {
        return null;
    } else {
        Node n = goal;
        SequentialPlan plan;
        for (plan = new SequentialPlan(); n.getParent() != null; n = n.getParent()) {
            Action action = (Action) codedProblem.getActions().get(n.getAction());
            plan.add(0, action);
        }
        return plan;
    }
}

```

Euristica

L'euristica è una componente cruciale nel miglioramento dell'efficienza degli algoritmi di ricerca nella pianificazione automatica. Nel progetto, è stata sviluppata un'euristica combinata che guida l'esplorazione dei nodi durante la ricerca, favorendo quelli che sono più promettenti per raggiungere l'obiettivo.

L'euristica sviluppata si basa su due approcci principali:

Fast Forward (FF):

- Questo approccio si basa sulla risoluzione di un problema rilassato, che semplifica il problema originale rimuovendo alcune restrizioni (ad esempio, trattando le azioni come indipendenti e ignorando gli effetti negativi). La soluzione del problema rilassato fornisce una stima del costo per raggiungere l'obiettivo dallo stato corrente.

Count Fluents:

- Questo approccio valuta quanti fluents dell'obiettivo non sono ancora stati soddisfatti nello stato corrente. In particolare, si considerano solo i fluents positivi, poiché gli obiettivi del nostro problema non includono letterali negativi. La logica dietro questo approccio è che uno stato più vicino all'obiettivo dovrebbe avere un numero inferiore di fluents insoddisfatti.

Implementazione del Codice

Algoritmi.java

La classe Algoritmi è il nucleo del sistema di pianificazione. Verifica se il problema è supportato e avvia il processo di ricerca per risolverlo. Include vari attributi per monitorare peso, timeout, nodi esplorati, memoria utilizzata, tempo di ricerca, il problema stesso, il tipo di euristica e il tipo di ricerca. I metodi principali includono l'istanziamento del problema e la verifica del supporto del problema.

HeuristicEvaluation.java

La classe HeuristicEvaluation, estendendo RelaxedGraphHeuristic, fornisce una stima del costo per raggiungere l'obiettivo utilizzando un grafo di pianificazione rilassato. L'euristica è calcolata combinando la risoluzione del problema rilassato e il conteggio dei fluents dell'obiettivo non soddisfatti nello stato corrente. I metodi principali includono la stima del costo per uno stato e la stima del costo per un nodo.

Conclusione

L'integrazione delle classi Algoritmi e HeuristicEvaluation rappresenta un sistema di pianificazione efficace basato su PDDL. Utilizzando tecniche di ricerca avanzate e

un'euristica combinata, il sistema è in grado di trovare soluzioni sub-ottimali in tempi ragionevoli, affrontando con successo problemi di pianificazione complessi.

Risultati dei test

A*

Istanza 1:

```
Actions: 170 Fluents: 63
Start a-star search. Timeout: 10

Found plan as follows:

00: (prendi a1 warehouse c1 b3 q0 q1) [0]
01: (   riempi a1 warehouse valve b3) [0]
02: (   muovi a1 warehouse l2 c1) [0]
03: (   svuota a1 l2 valve b3 p3) [0]
04: (   muovi a1 l2 warehouse c1) [0]
05: (   riempi a1 warehouse valve b3) [0]
06: (   muovi a1 warehouse l3 c1) [0]
07: (   svuota a1 l3 valve b3 p4) [0]
08: (   muovi a1 l3 warehouse c1) [0]
09: (   riempi a1 warehouse tool b3) [0]
10: (   muovi a1 warehouse l3 c1) [0]
11: (   svuota a1 l3 tool b3 p4) [0]
12: (   muovi a1 l3 warehouse c1) [0]
13: (   riempi a1 warehouse tool b3) [0]
14: (   muovi a1 warehouse l1 c1) [0]
15: (   svuota a1 l1 tool b3 p1) [0]
16: (   muovi a1 l1 warehouse c1) [0]
17: (   riempi a1 warehouse bolt b3) [0]
18: (   muovi a1 warehouse l1 c1) [0]
19: (   svuota a1 l1 bolt b3 p2) [0]
20: (   muovi a1 l1 warehouse c1) [0]
21: (   riempi a1 warehouse bolt b3) [0]
22: (   muovi a1 warehouse l1 c1) [0]
23: (   svuota a1 l1 bolt b3 p1) [0]

Time spent for searching:    2,71 seconds
Nodes Explored: 36095
Memory used for search:    21,80 MBytes
```

Istanza 2:

```
Actions: 768 Fluents: 90
Start a-star search. Timeout: 10

No plan found

Time spent for searching:    600,05 seconds
Nodes Explored: 1341805
Memory used for search:    1566,40 MBytes
```

Weighted A*

Istanza 1:

```
Actions: 170 Fluents: 63  
Start weighted a-star search. Weight: 1.5 Timeout: 10
```

Found plan as follows:

```
00: (prendi a1 warehouse c1 b2 q0 q1) [0]  
01: (    riempi a1 warehouse valve b2) [0]  
02: (    muovi a1 warehouse l2 c1) [0]  
03: (    svuota a1 l2 valve b2 p3) [0]  
04: (    muovi a1 l2 warehouse c1) [0]  
05: (    riempi a1 warehouse valve b2) [0]  
06: (    muovi a1 warehouse l3 c1) [0]  
07: (    svuota a1 l3 valve b2 p4) [0]  
08: (    muovi a1 l3 warehouse c1) [0]  
09: (    riempi a1 warehouse bolt b3) [0]  
10: (    riempi a1 warehouse tool b2) [0]  
11: (    muovi a1 warehouse l3 c1) [0]  
12: (    svuota a1 l3 tool b2 p4) [0]  
13: (    muovi a1 l3 warehouse c1) [0]  
14: (    riempi a1 warehouse tool b2) [0]  
15: (    muovi a1 warehouse l1 c1) [0]  
16: (    svuota a1 l1 tool b2 p1) [0]  
17: (    muovi a1 l1 warehouse c1) [0]  
18: (    riempi a1 warehouse bolt b2) [0]  
19: (    muovi a1 warehouse l1 c1) [0]  
20: (    svuota a1 l1 bolt b2 p1) [0]  
21: (    muovi a1 l1 warehouse c1) [0]  
22: (    riempi a1 warehouse bolt b2) [0]  
23: (    muovi a1 warehouse l1 c1) [0]  
24: (    svuota a1 l1 bolt b2 p2) [0]
```

```
Time spent for searching:    0,20 seconds  
Nodes Explored: 362  
Memory used for search:    0,46 MBytes
```

Istanza 2:

Actions: 768 Fluents: 90

Start weighted a-star search. Weight: 1.5 Timeout: 10

Found plan as follows:

```
00: (prendi a1 warehouse c1 b2 q1 q2) [0]
01: (   riempi a1 warehouse valve b2) [0]
02: (     muovi a1 warehouse l2 c1) [0]
03: (     svuota a1 l2 valve b2 p3) [0]
04: (   muovi-agente a1 l2 warehouse) [0]
05: (prendi a1 warehouse c2 b1 q1 q2) [0]
06: (prendi a1 warehouse c2 b3 q0 q1) [0]
07: (   riempi a1 warehouse bolt b3) [0]
08: (   riempi a1 warehouse tool b1) [0]
09: (     muovi a1 warehouse l4 c2) [0]
10: (     svuota a1 l4 bolt b3 p6) [0]
11: (     svuota a1 l4 tool b1 p6) [0]
12: (     muovi a1 l4 warehouse c2) [0]
13: (   riempi a1 warehouse valve b3) [0]
14: (   riempi a1 warehouse tool b1) [0]
15: (     muovi a1 warehouse l4 c2) [0]
16: (     svuota a1 l4 valve b3 p6) [0]
17: (     muovi a1 l4 l3 c2) [0]
18: (     svuota a1 l3 tool b1 p5) [0]
19: (     muovi a1 l3 warehouse c2) [0]
20: (   riempi a1 warehouse bolt b1) [0]
21: (   riempi a1 warehouse tool b3) [0]
22: (     muovi a1 warehouse l1 c2) [0]
23: (     svuota a1 l1 bolt b1 p2) [0]
24: (     svuota a1 l1 tool b3 p1) [0]
25: (     muovi a1 l1 warehouse c2) [0]
26: (   riempi a1 warehouse bolt b1) [0]
27: (   riempi a1 warehouse tool b3) [0]
28: (     muovi a1 warehouse l3 c2) [0]
29: (     svuota a1 l3 tool b3 p4) [0]
30: (     svuota a1 l3 bolt b1 p5) [0]
31: (     muovi a1 l3 warehouse c2) [0]
32: (   riempi a1 warehouse bolt b1) [0]
33: (   riempi a1 warehouse valve b3) [0]
34: (     muovi a1 warehouse l1 c2) [0]
35: (     svuota a1 l1 bolt b1 p1) [0]
36: (     muovi a1 l1 l3 c2) [0]
37: (     svuota a1 l3 valve b3 p4) [0]
38: (     muovi a1 l3 warehouse c2) [0]
39: (   riempi a1 warehouse valve b1) [0]
40: (     muovi a1 warehouse l3 c2) [0]
41: (     svuota a1 l3 valve b1 p5) [0]
```

Time spent for searching: 2,92 seconds

Nodes Explored: 4083

Memory used for search: 8,06 MBytes

Enforced Hill Climbing

Istanza 1:

Actions: 170 Fluents: 63

Found plan as follows:

```
00: (prendi a1 warehouse c1 b1 q0 q1) [0]
01: (      riempi a1 warehouse bolt b1) [0]
02: (      muovi a1 warehouse l1 c1) [0]
03: (      svuota a1 l1 bolt b1 p1) [0]
04: (      muovi a1 l1 warehouse c1) [0]
05: (      riempi a1 warehouse bolt b1) [0]
06: (      muovi a1 warehouse l2 c1) [0]
07: (      muovi a1 l2 l1 c1) [0]
08: (      svuota a1 l1 bolt b1 p2) [0]
09: (      muovi a1 l1 warehouse c1) [0]
10: (      riempi a1 warehouse valve b1) [0]
11: (      muovi a1 warehouse l1 c1) [0]
12: (      muovi a1 l1 l2 c1) [0]
13: (      svuota a1 l2 valve b1 p3) [0]
14: (      muovi a1 l2 l1 c1) [0]
15: (      muovi a1 l1 warehouse c1) [0]
16: (      riempi a1 warehouse valve b1) [0]
17: (      muovi a1 warehouse l1 c1) [0]
18: (      muovi a1 l1 l3 c1) [0]
19: (      svuota a1 l3 valve b1 p4) [0]
20: (      muovi a1 l3 warehouse c1) [0]
21: (      riempi a1 warehouse tool b1) [0]
22: (      muovi a1 warehouse l1 c1) [0]
23: (      svuota a1 l1 tool b1 p1) [0]
24: (      muovi a1 l1 l3 c1) [0]
25: (      muovi a1 l3 warehouse c1) [0]
26: (      riempi a1 warehouse tool b1) [0]
27: (      muovi a1 warehouse l3 c1) [0]
28: (      svuota a1 l3 tool b1 p4) [0]
```

Time spent for searching: 0,13 seconds

Nodes Explored: 263

Memory used for search: 0,09 MBytes

Istanza 2:

Actions: 768 Fluents: 90

Found plan as follows:

```
00: (prendi a1 warehouse c1 b1 q1 q2) [0]
01: (prendi a1 warehouse c1 b2 q0 q1) [0]
02: (    riempi a1 warehouse bolt b1) [0]
03: (    muovi a1 warehouse l1 c1) [0]
04: (    svuota a1 l1 bolt b1 p1) [0]
05: (    muovi a1 l1 warehouse c1) [0]
06: (    riempi a1 warehouse bolt b1) [0]
07: (    muovi a1 warehouse l1 c1) [0]
08: (    svuota a1 l1 bolt b1 p2) [0]
09: (    muovi a1 l1 warehouse c1) [0]
10: (    riempi a1 warehouse bolt b1) [0]
11: (    muovi a1 warehouse l3 c1) [0]
12: (    svuota a1 l3 bolt b1 p5) [0]
13: (    muovi a1 l3 warehouse c1) [0]
14: (    riempi a1 warehouse bolt b1) [0]
15: (    muovi a1 warehouse l4 c1) [0]
16: (    svuota a1 l4 bolt b1 p6) [0]
17: (    muovi a1 l4 warehouse c1) [0]
18: (    riempi a1 warehouse valve b1) [0]
19: (    riempi a1 warehouse tool b2) [0]
20: (    muovi a1 warehouse l1 c1) [0]
21: (    svuota a1 l1 tool b2 p1) [0]
22: (    muovi a1 l1 l2 c1) [0]
23: (    svuota a1 l2 valve b1 p3) [0]
24: (    muovi a1 l2 l3 c1) [0]
25: (    muovi a1 l3 warehouse c1) [0]
26: (    riempi a1 warehouse valve b1) [0]
27: (    riempi a1 warehouse tool b2) [0]
28: (    muovi a1 warehouse l3 c1) [0]
29: (    svuota a1 l3 valve b1 p4) [0]
30: (    svuota a1 l3 tool b2 p4) [0]
31: (    muovi a1 l3 warehouse c1) [0]
32: (    riempi a1 warehouse valve b1) [0]
33: (    riempi a1 warehouse tool b2) [0]
34: (    muovi a1 warehouse l3 c1) [0]
35: (    svuota a1 l3 valve b1 p5) [0]
36: (    svuota a1 l3 tool b2 p5) [0]
37: (    muovi a1 l3 l4 c1) [0]
38: (    muovi a1 l4 warehouse c1) [0]
39: (    riempi a1 warehouse valve b1) [0]
40: (    riempi a1 warehouse tool b2) [0]
41: (    muovi a1 warehouse l4 c1) [0]
42: (    svuota a1 l4 valve b1 p6) [0]
43: (    svuota a1 l4 tool b2 p6) [0]
```

Time spent for searching: 0,36 seconds

Nodes Explored: 163

Memory used for search: 0,34 MBytes

Task 3

3.1

Il terzo task del progetto prevede la conversione del PDDL all'utilizzo di azioni durative. A partire dal domain classico, è stato introdotto un ulteriore predicato **libero_agente** che ci permette di definire quando un agente è libero di svolgere una determinata azione. Questo impedisce l'eventuale svolgimento di azioni in parallelo.

```
(define (domain dominio)
  (:requirements :typing :strips :durative-actions :negative-preconditions)
  (:types
    location locatable - object
    box agent carrier workstation content - locatable
    quantity - object
  )

  (:predicates
    (box_vuota ?b - box)
    (capacita ?c - carrier ?cn - quantity)
    (pos ?x - locatable ?v - location)
    (nella_scatola ?b - box ?ct - content)
    (servito ?p - workstation ?c - content)
    (sul_carrello ?b - box ?c - carrier)
    (carrello_vuoto ?c - carrier)
    (precedente ?cn1 ?cn2 - quantity)
    (libero_agente ?a - agent )
  )
)
```

Muovi e muovi_agente

Per l'azione **muovi** è stata scelta una durata arbitraria di 5 secondi, mentre per l'azione **muovi agente** è stata scelta una durata di 1 secondo. A differenza del domain classico, è necessario specificare quando una determinata condizione o effetto è valido temporalmente. Il costrutto **at start** indica che la condizione deve verificarsi all'inizio dell'azione, mentre il predicato **at end** indica che l'azione deve verificarsi alla fine.

```
(:durative-action muovi
:parameters (?a - agent ?l1 ?l2 - location ?c - carrier)
:duration ( = ?duration 5)
:condition (and
  (at start (libero_agente ?a))
  (at start (pos ?c ?l1))
  (at start (pos ?a ?l1))
  (at start (not (carrello_vuoto ?c)))
)
:effect (and
  (at start(not(libero_agente ?a)))
  (at start(not (pos ?c ?l1)))
  (at start(not (pos ?a ?l1)))
  (at end(pos ?c ?l2))
  (at end(pos ?a ?l2))
  (at end(libero_agente ?a))
)
)
```

```
(:durative-action muovi_agente
:parameters (?a - agent ?l1 ?l2 - location)
:duration ( =?duration 1)
:condition (and
  (at start(libero_agente ?a))
  (at start(pos ?a ?l1))
)
:effect (and
  (at start(not(libero_agente ?a)))
  (at start(not (pos ?a ?l1)))
  (at end(pos ?a ?l2))
  (at end(libero_agente ?a))
)
)
```

Prendi e toglì

Sia per l'azione **prendi** che per l'azione **togli** è stata scelta una durata arbitraria di 2 secondi. Analoghe alle precedenti, con l'unica differenza che in questo caso è presente anche il costrutto **over all** che indica che una determinata condizione deve essere verificata per tutta la durata dell'azione.

```
(:durative-action prendi
:parameters (?a - agent ?l - location ?c - carrier ?b - box ?q1 ?q2 - quantity)
:duration (=?duration 2)
:condition (and
  (at start(libero_agente ?a))
  (over all (pos ?c ?l))
  (over all (pos ?a ?l))
  (at start (pos ?b ?l))
  (at start(capacita ?c ?q2))
  (at start(precedente ?q1 ?q2))
)
:effect (and
  (at start(not(libero_agente ?a)))
  (at start(not (pos ?b ?l)))
  (at end(capacita ?c ?q1))
  (at end(sul_carrello ?b ?c))
  (at end(not (capacita ?c ?q2)))
  (at end(not (carrello_vuoto ?c)))
  (at end(libero_agente ?a))
)
)
```

```
(:durative-action toglì
:parameters (?a - agent ?l - location ?c - carrier ?b - box ?q1 ?q2 - quantity)
:duration (=?duration 2)
:condition (and
  (at start(libero_agente ?a))
  (over all(pos ?c ?l))
  (over all(pos ?a ?l))
  (at start(capacita ?c ?q1))
  (at start(sul_carrello ?b ?c))
  (at start(precedente ?q1 ?q2))
)
:effect (and
  (at start(not(libero_agente ?a)))
  (at end(pos ?b ?l))
  (at end(capacita ?c ?q2))
  (at end(not (capacita ?c ?q1)))
  (at end(not (sul_carrello ?b ?c)))
  (at end(libero_agente ?a))
)
)
```

Riempi e svuota

Alle azioni **riempi** e **svuota**, è stata assegnata una durata arbitraria di 3 secondi. Queste due azioni hanno subito una modifica anche nella logica, rispetto al dominio classico: questo è dovuto al fatto che alcuni planner di plansys non supportano i costrutti esistenziali o disgiuntivi (**exists**, **or**) che evitavano nel dominio classico alcuni viaggi “inutili” dell'agente.

```
(:durative-action riempi
:parameters (?a - agent ?l - location ?c - content ?b - box)
:duration (=?duration 3)
:condition (and
  (at start(libero_agente ?a))
  (over all(pos ?a ?l))
  (over all(pos ?c ?l))
  (over all(pos ?b ?l))
  (at start(box_vuota ?b))
)
:effect (and
  (at start(not (libero_agente ?a)))
  (at end(not (box_vuota ?b)))
  (at end(nella_scatola ?b ?c))
  (at end(libero_agente ?a))
)
)
```

```
(:durative-action svuota
:parameters (?a - agent ?l - location ?c - content ?b - box ?p - workstation)
:duration (=?duration 3)
:condition (and
  (at start(libero_agente ?a))
  (over all(pos ?a ?l))
  (over all(pos ?p ?l))
  (at start(not (box_vuota ?b)))
  (at start(nella_scatola ?b ?c))
  (over all(pos ?b ?l))
)
:effect (and
  (at start(not(libero_agente ?a)))
  (at end(servito ?p ?c))
  (at end(box_vuota ?b))
  (at end(not (nella_scatola ?b ?c)))
  (at end(libero_agente ?a))
)
)
```

Nodi ROS

Ad ogni azione del dominio è stato associato un nodo ROS. L'implementazione dei nodi ROS è all'interno di file *cpp*. Nel nostro caso i nodi non svolgono alcuna azione particolare, sono più che altro fittizi: incrementano un contatore e ne controllano il valore fino al completamento.

```

class MoveAction : public plansys2::ActionExecutorClient
{
public:
    MoveAction()
    : plansys2::ActionExecutorClient("move", 1s)
    {
        progress_ = 0.0;
    }

private:
    void do_work()
    {
        if (progress_ < 1.0) {
            progress_ += 0.02;
            send_feedback(progress_, "Move running");
        } else {
            finish(true, 1.0, "Move completed");

            progress_ = 0.0;
            std::cout << std::endl;
        }

        std::cout << "\r\e[K" << std::flush;
        std::cout << "Moving ... [" << std::min(100.0, progress_ * 100.0) << "%] " <<
            std::flush;
    }
}

```

Tutti i nodi vengono messi in esecuzione in ROS dal launcher presente nella cartella **launch**.

Per la generazione del piano con plansys2 è necessario utilizzare il comando:
planutils run lpg-td domain.pddl ist2.pddl

Utilizzando il precedente domain e l'istanza opportunamente modificata.